

整理模块重构 PRD

可配置整理模板

大模型根据用户的要求，把用户自己的记忆记录整理成便于自己查看的数据 —— 要求从代码里拿出来，产物从"给人读的长文"变成"给自己查的数据"。

项目	睿乐大脑 · 整理模块 (Organize)
版本	v1.4 (待评审)
对标现状	v0.7.0 / internal/application/service/organize*.go
覆盖范围	记忆 · 成果 · 发现 三个页签的 AI 整理能力
模板维护方	平台 · admin 后台 · 资产治理
产物受众	记录者本人 (self)

0

代码中的 AI 提示词字面量

8+4

整理模板 (8 场景 + 4 存量迁移)

50

单次可整理记忆条数上限

整理模块重构 PRD —— 可配置整理模板

版本：v1.4（待评审） 一句话描述：大模型根据用户的要求，把用户自己的记忆记录整理成便于自己查看的数据。 范围：整理 模块（记忆 / 成果 / 发现）的 AI 整理能力重构 关键词：整理要求、整理模板、模板驱动、指令可配置、产物可筛可查、可回溯、可灰度 v1.1 变更：模板维护方确定 —— 由平台在 admin 后台维护和创建。原 v1.0 的待决策项 Q1/Q2 结题，连带调整模板真源、运行期读取方式、权限模型与分期节奏。 v1.2 变更：确立产品定义 —— "大模型根据用户的要求整理用户的记忆数据"。据此把引擎入口抽象为「整理要求」，明确"要求"的两种承载形态（预置要求 = 模板 / 自定义要求 = 自由描述），并新增 Q8 待拍板。 v1.4 变更：补齐路由与深链设计 (§10.6) —— 页签扩为 4 条（新增工作台）、新增 2 条二级路由（整理详情时间线 / 产物查看）、约定 4 条深链 query、明确弹窗不路由的边界；配套补 §9.2.2 整理配置接口组、AC-21/22，并新增 Q10（默认整理直通）、Q11（redirect 落点）。 v1.3 变更：定义收敛到产物形态与受众 —— "整理成便于自己查看的数据"。据此新增「查看契约」（ audience / fields / view ）、§5.5「便于查看」四条硬要求、跳回原文的引用锚点，并把「成果」页签的边界提为 Q9 待确认。

阅读导航

章节	回答的问题	读者
1. 执行摘要	这次要做什么、能带来什么	全员
2. 背景与现状诊断	现在为什么不行	研发 / 产品
3. 目标与边界	做什么、不做什么、关键决策	全员
4. 核心概念：整理模板	"整理模板"到底是什么	产品 / 研发 / 运营
5. 模板配置格式	一条整理指令由哪些字段构成	平台运营 / 研发
6. 数据模型设计	表怎么建	研发
7. 执行管线	一次整理从点击到落库发生了什么	研发
8. 功能需求（EARS）	验收口径	测试 / 产品
9. API 契约	接口怎么调	前端 / 研发
10. 界面与交互	前台与平台后台分别长什么样	前端 / 设计 / 平台运营
11. 兼容与迁移	老数据、老行为怎么办	研发
12. 分期实施	先做什么	全员
13. 风险与验收标准	怎么算成功	全员
10.6 路由与深链	页面地址怎么定、链接怎么分享	前端 / 产品
14. 待决策问题	已定的事 + 还要拍板的 9 件事	负责人

1. 执行摘要

1.1 一句话

大模型根据用户的要求，把用户自己的记忆记录整理成便于自己查看的数据。

这句话拆开来看，每个词都在限定设计的一半：

词	含义	对应的设计约束
用户的要求	整理成什么、按什么口径	前台必须有"表达要求"的入口；要求必须能被结构化落定之后才交给模型
大模型	执行者，不是决策者	模型只按已确定的要求执行；口径模糊、要求缺失时不得由模型自由发挥
自己的记忆记录	输入与权限边界	只取记录者本人有权访问的记忆；跨用户取数是越权，不是功能
便于自己查看	产物的读者是记录者本人	优化目标是"自己回头能快速找到、看懂、用上"，不是"给别人看的完整叙述"
数据	产物的形态	产物不能只是一篇长文：必须有可筛选、可排序、可定位的结构化字段，正文只是它的展开

「便于自己查看的数据」是全篇最容易被做偏的一句。 它的反面就是现状：发芽报告是一篇 3 段式的长文，读者要通读才能取用；没有标签、没有时间轴、没有字段，回头想找"上次关于影子的那次磨课"只能靠肉眼翻。"便于查看"不是排版问题，是数据结构问题。

这句话排除了什么（同样重要）

不是	因为
不是替用户写材料	产物读者是自己，不需要面向他人的措辞包装（写给家长的沟通稿、上报的汇报材料不在本期）
不是替用户做分析决策	定义里没有"给出建议""判断优劣"；需要多步推理的场景属于 Agent 模块
不是把记忆压缩成摘要	摘要是"更短的长文"，仍然是长文；要求的是"可查的结构"
不是无中生有的创作	输入是用户自己的记录，产物必须可回溯到具体记忆，不得编造

「用户的要求」有两种承载形态，这是本设计的关键分叉：

形态	载体	用户如何表达	本期
预置要求	整理模板（平台在 admin 后台配好的整理方案）	从方案里选一个，再指定范围	✅ 主体
自定义要求	用户自由描述（"把这周的家长沟通记录整理成复盘"）	自然语言	⌚ 见 Q8

产品定义 ≠ 实现手段。 v1.0 曾把"运营/教研提前把指令写进模板"直接当成定义来写，那是**实现手段**。这一版把两者分开：定义是"大模型按用户要求整理用户记忆数据"，模板只是当下承载"要求"的**主要手段**，不是唯一手段。

重构前真正的病根也不是"没有要求"，而是**要求被写死在代码里**：口径只有一种、用户无法表达、改口径要改 Go 代码并全量发版。

1.2 现状与目标的对比

维度	现状 (v0.7.0)	目标
用户能否表达"要求"	不能。口径写死在代码里，只有一种	能：选预置方案 + 在方案上限内定范围（自定义要求见 Q8）
要求是否被落定	无所谓"要求"，只有一段拼接好的 prompt	落定为不可变快照，与 job 一起冻结（D11）
产物第一读者	不明确，实际按"读起来完整"做	记录者本人：优化目标是回查效率，不是叙述完整度
产物形态	一篇长文（3 段式报告），要通读才能取用	结构化字段 + 正文展开：可筛选、可排序、可跳回原文
整理指令存放位置	Go 源码字符串常量（4 处）	数据库（唯一真源），由平台在 admin 后台维护创建
模板维护入口	无（只能改代码）	admin 后台 → 「资产治理」 → 「整理模板」
新增一个整理场景	改代码 → 发版 → 全量生效	后台新增模板 → 试跑 → 发布 → 即时生效
调整一句指令	改 Go 代码	后台改模板文本 → 试跑 → 发布，可回滚
改动对线上的影响	一改全量生效	草稿与已发布快照分离，发布才生效
可治理性	无版本、无人负责	版本快照、发布留痕、按版本回滚
输出结构	写死「3 个二级标题 + 🌱 种子 + ✨ Aha」	模板声明章节契约，可选、可校验
输入范围	单条记忆（发芽） / 单个文件（成果）	模板声明筛选条件，支持 多条记忆聚合整理
场景分类	8 个分类双份硬编码（后端 + 前端）	由模板 <code>scene</code> 字段驱动，接口下发
模型参数	<code>temperature/max_tokens</code> 写死	模板级配置
可观测性	仅有 <code>ai_status: completed/fallback</code>	任务级状态机 + 模板效果看板
可回溯性	无	产物记录 <code>template_key + version + prompt_hash + model_id</code>

1.3 预期收益

- 1. **平台自助**：平台运营在 admin 后台即可新建、调整、停用整理指令，不再排研发需求、不用发版。

- 2. 场景可扩展：从当前 1 个"发芽报告"场景扩展到 8 个园所经营场景，全部是后台配置工作量。
- 3. 质量可度量：按模板与版本统计"成功率 / 降级率 / 用户采纳率"，用数据迭代指令。
- 4. 风险可控：草稿与发布分离 + 版本快照 + 启用开关，指令改动可试跑、可回滚、可停用。
- 5. 回查效率提升：产物带结构化字段与引用锚点，用户从"翻长文找"变成"筛一下 + 跳回去"，整理结果真正能被自己用上。

2. 背景与现状诊断

2.1 代码事实盘点

整理模块当前的实现分布（均为本次重构对象）：

文件	行数	职责
internal/application/service/organize.go	633	记忆 / 成果 / 发芽的 CRUD、Overview
internal/application/service/organize_upload.go	789	上传解析、3 处硬编码 AI 提示词、模型解析
internal/application/service/organize_sprout.go	417	发芽报告生成、1 处硬编码提示词 + 骨架降级文案
internal/application/service/organize_memory_audio.go	549	音频入库、ASR 转写任务
internal/application/service/organize_discover.go	383	发现页聚合
internal/types/organize.go	312	实体定义 + 8 个分类常量硬编码
internal/types/interfaces/organize.go	61	仓储 / 服务接口
internal/router/router.go:1354	—	22 条 /organize/* 路由
frontend/src/views/organize/*	—	工作台、编辑器、分类常量第二份硬编码
migrations/versioned/000072_organize_section.up.sql	—	4 张表

整理模块 AI 能力现状：只有 1 条真正的"整理"链路 —— 单条记忆 → 发芽报告；其余 3 条是"元数据生成"（导入记忆、录音笔记、成果卡片）。

2.2 七个结构性缺口

缺口 1：「用户的要求」没有承载物

buildSproutAIPrompt() 把写作要求直接拼接进 Go 字符串：

```
// internal/application/service/organize_sprout.go:193-226 (节选)
return fmt.Sprintf(`请基于一条记忆生成一份"发芽报告"。
...
- 至少包含 3 个二级标题，使用"## 01. 标题"格式。
- 每个部分都要包含"🌱 种子"和"✨ Aha 瞬间"两个小段落。
...`)
```

四处 AI 提示词均为同一形态：`fmt.Sprintf` + 中文字面量。

位置	用途	输出结构
<code>organize_sprout.go:193</code>	发芽报告	强制 Markdown 章节结构
<code>organize_upload.go:227</code>	导入记忆元数据	JSON <code>{title, summary, tags}</code>
<code>organize_upload.go:292</code>	录音转写笔记	JSON <code>{..., note_markdown}</code>
<code>organize_upload.go:363</code>	成果卡片元数据	JSON <code>{title, summary, tags}</code>

后果：定义里的"用户的要求"在系统里根本无处安放 —— 用户只能接受唯一口径，无法表达自己的诉求；调一句"语气再温和些"要排期发版；园所教研专家无法参与指令打磨；不同园所无法有不同口径。

缺口 2：一个模板打天下

所有场景（招生、家长沟通、教研、食育、环创……）都被压缩进同一份"发芽报告"指令，`## 01. / 🌱 种子 / ✨ Aha` 是强制结构。而"招生线索跟进清单"和"教研成果提炼"显然需要不同的输出形态。

缺口 3：输出结构不可校验、不可扩展

`generateSproutReportFromMemory()` 拿到模型返回后只做 `TrimSpace`，不校验是否真的包含 3 个二级标题、是否包含 `🌱 / ✨`。模型跑偏时用户拿到的是不合规内容，且系统无从知晓。

缺口 4：输入范围过窄

`CreateSproutReportFromMemory` 只接受单个 `memory_id`。但真实整理需求是"把这周所有家长沟通的记录整理成一份复盘"—— 需要按 `kind / tag / 时间范围` 批量取数并做预算控制。当前没有任何批量装配能力。

缺口 5：分类硬编码双份

```
// internal/types/organize.go:217-239
OrganizeDiscoverCategoryAdmissionsGrowth = "admissions_growth"
... // 共 8 个

// frontend/src/views/organize/discoverCategories.ts
export const DISCOVER_CATEGORIES = [ { key: 'admissions_growth', label: '招生增长' }, ... ]
```

后端改分类要同步改前端，还要补 `categoryAliases` 兼容旧值。

缺口 6：无版本、无灰度、无效果回收

- 产物上不记录"用了哪份指令的哪一版"，指令迭代后无法归因效果。
- 没有灰度机制，一次指令改动全量生效，出问题只能回滚代码。
- `ai_status` 只有 `completed / fallback` 两态，无法区分"模型跑偏"和"模型未配置"。
- 降级文案是硬编码骨架（`organize_sprout.go:228-281`），不可配置、不可按模板定制。

缺口 7：产物是"文章"，不是"便于查看的数据"

发芽报告与成果卡片的产物落库时只有一块 Markdown 正文：**没有可筛选的字段、没有可排序的时间轴、没有可跳回的锚点。**

后果：用户回头想找"上次关于影子的那次磨课"，只能逐条点开长文肉眼翻。整理是做完了，但**取用成本没降**——这与"便于自己查看"的目标正好相反。这个缺口不会被"换个更好的提示词"修好，它是**数据结构**上的缺失。

2.3 为什么不能靠"加字段"解决

有人会提出："加个 `custom_prompt` 字段不就行了？"

不行，原因有三：

1. **拼字符串不可控。**用户/运营写 `{{memory.content}}` 之外的任何变量都会让渲染静默失败；没有变量白名单、没有长度预算、没有注入防护。
2. **缺契约就没有质量下限。**没有"输出契约 + 门禁 + 自修复"的设计，模板越自由，产出越不可控，最终用户会觉得"AI 整理不好用"。
3. **缺版本就没有迭代能力。**指令是可迭代资产，必须版本化、可对比、可回滚、可度量，否则改一次坏一次。

所以本次是**架构重构**，不是字段扩展。

3. 目标与边界

3.1 业务目标

编号	目标	衡量口径
G1	整理指令全部配置化	AI 整理相关提示词在 Go 代码中 0 处字面量
G2	场景可自助扩展	平台管理员在 admin 后台新增一个整理场景 ≤ 30 分钟，无需发版、无需重启服务
G3	整理能力从"单条"升级为"批量"	支持一次整理 ≤ 50 条记忆
G4	输出质量可校验	100% 产物经过模板契约门禁
G5	效果可归因	每个产物可追溯 <code>template_key + version + prompt_hash + model_id</code>
G6	行为可平滑迁移	存量 4 条 AI 链路行为等价，可灰度回退
G7	模板可治理	平台后台可查版本、可回滚、变更留痕、可一键停用
G8	用户要求可表达、可落定	用户无需理解"提示词"概念，即可在前台表达"要整理成什么"；100% 的整理任务在调用模型前已落定为一组结构化要求
G9	产物便于自己查看	每个产物都带结构化字段（可筛选、可排序）；任一条结论可一跳回到原始记忆；不需要通读全文即可取用

3.2 本期范围

包含

- 「整理要求」的领域抽象：要求的落定、校验与传递（模板是其中一种要求提供方）
- 「查看契约」：产物的结构化字段、查看视图与跳回原文的引用锚点（`output.audience / fields / view`）
- 整理模板的领域模型、配置格式、加载与版本机制
- 模板驱动的整理执行管线（要求落定 → 输入装配 → 渲染 → 调用 → 解析 → 门禁 → 落库 / 降级）
- 平台后台模板管理：列表 / 新建 / 编辑 / 试跑 / 发布 / 停用 / 版本回滚 / 效果看板
- 模板首次初始化种子（含存量 4 条链路的等价迁移）与 YAML 导入导出
- 多记忆批量整理
- 存量 4 条 AI 链路切换到模板驱动（行为等价）
- 场景模板首批 8 条（对齐现有 8 个分类，均在后台创建）
- 模板选择接口 + 整理任务接口
- 前端：模板选择器、整理工作台、"用模板整理"入口

不包含（本期非目标）

- 不做面向他人的材料生成：产物受众固定 `audience: self`（记录者本人）；写给家长 / 上级 / 评估用的沟通稿、汇报材料不在本期（见 Q9）

- 不做分析、建议、决策：不给"孰优孰劣""下一步该怎么做"的结论性意见；这类多步推理需求走 Agent 模块
- 不做自定义要求（用户自由文字描述要求）：引擎预留 `requirement.provider = custom` 插槽，但本期不开放入口 —— 见 Q8
- 不做租户级 / 个人级模板自建（`visibility` 仅 `platform`，字段预留但本期只走平台层）
- 不做模板审核流（发布为单步操作 + 二次确认，靠版本回滚兜底）
- 不做模板可视化拖拽编辑器（先做分区表单 + 变量插入按钮）
- 不做模板市场 / 跨租户模板交易
- 不做多轮对话式整理（本期是一次生成 + 可重跑）
- 不做模型自动路由 / 成本最优选型（仅支持模板指定 + 系统默认）
- 不做自定义输出渲染器插件（渲染器 key 由内置集合枚举）
- 不改动发现页现有内容分发逻辑（仅把分类常量改为接口下发）

3.3 关键设计决策

编号	决策	方案	理由
D1	模板真源位置	DB 为唯一真源，平台管理员在 admin 后台维护创建。 <code>config/organize_templates/seed/*.yaml</code> 仅作首次初始化种子与导入导出格式，运行期不读文件	双真源必然漂移 —— <code>config/agent_type_presets.yaml</code> 的注释里团队已经明确记过这个教训（ <i>"avoid the two sources of truth drift that used to plague this file"</i> ）。只有把真源放进后台，才能做到"改指令不发版"
D2	运行期读哪份配置	读已发布快照（ <code>published_version</code> 指向的版本记录），不读编辑态 <code>spec</code>	编辑中的模板不能影响线上；发布与生效解耦，草稿可以随便改、随便试跑
D3	模板维护权限	平台系统管理员（ <code>requiresSystemAdmin: true</code> ），入口挂 admin 后台「资产治理」	与「智能体」的治理模型完全同构（后台描述即"统一维护平台内置智能体"）；前台用户只消费、不生产模板
D4	模板粒度	按「场景 × 产物形态」	招生增长场景产出"跟进清单"，教研场景产出"提炼稿"——形态由场景决定，分开配置更直观
D5	指令自由度的边界	模板可写指令文本，但变量必须来自白名单	自由度给到"文字"，不给到"数据通路"，避免渲染失败与注入
D6	输出契约	模板声明 + 服务端强制校验	保证质量下限；不达标先自修复一次，再降级
D7	执行模型	同步建任务 + 异步生成（沿用现有 <code>go s.complete...</code> 模式，升级为 <code>asynq</code> 任务）	大模型耗时长；现有发芽报告已是异步补齐，沿此习惯最平滑
D8	灰度和回退	模板级启用/停用开关 + 租户白名单灰度；库中模板不可用时回退到代码内置基线	存量链路不能因为配置缺失或后台误操作而不可用
D9	模板加载与缓存	进程内缓存，发布时主动失效 + TTL 30s 兜底	模板是"高频读、极低频写"；多实例部署下用短 TTL 兜住一致性，不引入重依赖
D10	兼容策略	先等价迁移，再扩展场景	先保证"行为不变"，再谈"能力变强"，降低回归风险
D11	"用户的要求"如何落地	要求先被结构化为 <code>OrganizeRequirement</code> 对象（目标产物 + 口径 + 范围 + 约束），再由引擎执行；模型不参与"要求是什么"的决策	定义里"大模型"是执行者。若要求不先落定就交给模型，等于让模型替用户定口径 —— 输出不可复现、不可归因、不可回滚

编号	决策	方案	理由
D12	要求的提供方	本期唯一提供方为平台预置模板； provider 字段预留 custom （用户自由描述）	先做可控形态，把"自由描述怎么落定成结构化要求"这道难题留到有真实需求数据之后（Q8）
D13	产物的读者与形态	audience 固定 self ；每个产物必须声明结构化字段（ fields ）与查看视图（ view ），正文只是字段的展开	定义里"便于自己查看的数据"。若只产出正文，回查效率与现状无异 —— 重构就白做了
D14	能否跳回原文	产物中的结论须带记忆引用锚点（ [M1] ），新场景模板默认 citation: required ；存量 4 条迁移例外保持 optional 以维持行为等价	"便于查看"的前提是能从整理结果一跳回到原始记录；但存量行为不能因此改变（D10 优先级更高）

3.4 术语约定（面向园所/教培行业）

技术术语	对用户呈现的表述
Requirement	整理要求 / 你想整理成什么
Preset requirement	预置方案（= 整理模板）
Custom requirement	自定义要求（本期不做）
Template	整理模板 / 整理方案
Render prompt	（不呈现）
Job / Pipeline	整理任务
Output contract	内容结构要求
Output	整理结果 / 我的整理（避免与「成果」页签名冲突）
View / Field	展示样式 / 可筛选的信息项（对用户不呈现术语，直接呈现控件）
Scene	经营场景
Fallback	基础版（生成失败时的保底内容）
Sprout report	发芽报告（沿用现有叫法）
Platform admin	平台管理员
Draft / Published	草稿 / 已发布
Version rollback	版本回滚
Seed template	初始化模板

4. 核心概念：整理模板

4.1 概念定义

整理要求 (Organize Requirement) —— 本模块的第一概念

定义里的**"用户的要求"**在系统内的落地形态。它是一组**已落定、可校验、可回放的**约束：产出什么形态、按什么口径、覆盖哪些记忆、不许出现什么。要求必须先落定、再执行 —— 这是 D11 的核心。落定后的要求与记忆 ID 列表一起构成一次整理的**完整输入**，模型只负责执行，不负责决定要求。

要求的四个要素：

要素	回答的问题	落定方式
① 目标产物	要产出什么（发芽报告 / 成果卡片 / 跟进清单...）	由要求的提供方声明
② 口径与结构	怎么组织内容、必须包含哪些章节	由提供方声明 + 平台约束
③ 作用范围	整理哪些记忆（类型、标签、时间、条数上限）	用户在界面上指定
④ 红线	不许编造、不许越权、不许超长	平台统一注入，不受提供方影响

整理模板 (Organize Template) —— 本期唯一的"要求提供方"

一份**预置好的整理要求**。它把某一类反复出现的要求固化下来：在什么场景下、拿哪些记忆、按什么指令、交给哪个模型、产出什么结构的内容、不达标怎么办。模板的价值在于**把个人经验沉淀成可复用的公共方案**，让用户不必每次重新描述要求。

维护方：平台。模板由平台管理员在 **admin 后台** → 「资产治理」→ 「整理模板」 维护和创建；前台用户只选择模板、不编辑模板。模板存在数据库里，运行期读"已发布快照"。

整理任务 (Organize Job)

一次要求的执行实例。记录落定后的要求、输入（记忆 ID 列表）、使用的模板版本、模型、状态、产出指针。

整理产物 (Organize Output)

任务的交付物，也是定义里**"便于自己查看的数据"**的落地形态。它不是一篇文档，而是一份**结构化记录 + 一段可展开的正文**：

组成	说明	回答的问题
字段 (fields)	可筛选、可排序、可聚合的结构化值：标题、摘要、标签、时间、涉及对象...	我能不能筛出来
视图 (view)	字段在界面上的呈现样式：列表行 / 卡片 / 时间轴 / 分组	我能不能扫过去
正文 (markdown)	字段的展开叙述，供需要细节时阅读	我想细看时有没有料
引用锚点 (citation)	结论 → 原始记忆的可点击锚点	我能不能跳回去
溯源 (provenance)	模板 key + 版本 + prompt_hash + 模型 + 输入记忆	这条结果怎么来的

一个产物如果只有"正文"没有"字段"，就不满足本定义 —— 那只是"更短的文章"，回头依旧要靠通读去找。字段是"便于查看"的承重墙。

模板七要素

要素	作用	缺省行为
① 触发条件 <code>trigger</code>	决定模板在哪些入口、哪些记忆类型下可选	所有入口、所有类型
② 输入装配 <code>input</code>	决定取哪些记忆、怎么排序、内容预算	显式选择的记忆，预算 7000 字
③ 指令 <code>instructions</code>	system + task 文本 + 变量白名单 + 示例 + 约束	必填
④ 模型策略 <code>model</code>	模型类型/ID、temperature、max_tokens、thinking、超时重试	租户默认模型
⑤ 输出契约 <code>output</code>	格式、目标实体、标题模板、必含章节、长度、引用要求	Markdown，无硬性章节
⑥ 质量门禁 <code>guardrails</code>	违规处置：自修复 / 降级 / 拒绝	自修复 1 次后降级
⑦ 降级策略 <code>fallback</code>	保底骨架、提示文案	使用内置通用骨架

4.2 实体关系

要求 → 模板 → 任务 的层次关系：要求是语义层，模板是要求的存储层，任务是一次执行。



4.3 与现有对象的关系映射

现有对象	重构后
OrganizeSproutReport	成为" target: sprout_report 模板"的产出容器，增加模板溯源字段
OrganizeOutput	成为" target: output 模板"的产出容器
OrganizeMemory.Metadata["tags"]	由 memory_meta 类模板产物写入
OrganizeDiscoverCategories()	由模板 scene 字段聚合生成，前端不再硬编码
4 处硬编码提示词	4 条 visibility: platform 模板，经初始化种子写入数据库

5. 模板配置格式（后台表单 / YAML 交换格式）

5.1 真源与文件约定

数据库是唯一真源，模板在 admin 后台维护和创建。YAML 只承担两个角色：① 首次初始化的种子；② 跨环境导入导出。

```
config/organize_templates/
├─ _schema.md                # 字段说明（给操作模板的人看）
├─ seed/                    # 初始化种子：仅在库中不存在该 key 时插入，永不覆盖
│   ├─ note_audio_transcribe.yaml # 录音转写 → 笔记（迁移自 organize_upload.go:292）
│   ├─ note_import_meta.yaml      # 导入记忆 → 元数据（迁移自 organize_upload.go:227）
│   ├─ output_card_meta.yaml      # 成果卡片 → 元数据（迁移自 organize_upload.go:363）
│   └─ sprout_review.yaml        # 发芽报告（迁移自 organize_sprout.go:193）
└─ fallbacks/                 # 降级骨架（Markdown，随包发布，不入库）
```

种子三条硬约束

- 1. 只插入缺失： `key` 已存在即跳过。运维在后台改过的内容不会被重启打回。
- 2. 可关闭： `organize.template_seed.enabled: false` 时整段逻辑不执行。
- 3. 不是运行期路径： `seed/` 里的文件改了对线上没有任何影响。要改线上，去后台。

走后台还是走种子

场景	走哪条路
首次部署 / 升级后补齐存量 4 条模板	种子自动写入
调整某条模板的指令	后台改 → 试跑 → 发布
把测试环境的模板同步到生产	后台「导出 YAML」 → 生产后台「导入」
新增一条经营场景模板（8 条这类）	后台新建，不进 <code>seed/</code>
新增一条要随版本发布的基线模板	加进 <code>seed/</code> ，对全新环境生效

8 条场景模板走**后台创建**，不进 `seed/` —— 它们是配置产物，不是代码资产。

5.2 完整模板 Schema

下面这份 YAML 是 `spec` 的规范描述。**后台编辑器的表单字段与它一一对应**，导入导出也用它 —— 后台字段与 YAML 字段是同一套定义，不留两套口径。

- tenant.role_label
- tenant.role_config
- user.nickname

③ 指令

instructions:

system: |

你是面向园所教研场景的记忆整理助手。输出必须是中文 Markdown，不输出 JSON。

task: |

请基于以下 `{{memory_count}}` 条记忆，生成一份「`{{template.name}}`」。

用户角色: `{{tenant.role_label}}`

角色关注点: `{{tenant.role_config}}`

时间范围: `{{memory_range}}`

写作要求:

- 先用 1 段话概括这批记忆共同指向的教研主题。
- 至少包含 3 个二级标题，使用"## 01. 标题"格式。
- 每个部分包含"🌱 观察"与"🌟 提炼"两个小段落。
- 最后给出 3 条可执行跟进行动。
- 不得编造记忆中没有的数字、人名、园所名。

记忆列表:

`{{memories}}`

variables:

显式声明白名单；未声明的变量渲染时报错

- template.name
- memory_count
- memory_range
- tenant.role_label
- tenant.role_config
- memories

examples:

few-shot, 可选

- input: |

[M1] 今天中班磨课，孩子对"影子"的追问超出预期.....

output: |

01. 儿童视角的意外发现

...

constraints:

拼进提示词的强约束清单

- 不得编造记忆中没有的事实
- 所有结论必须可回溯到具体记忆

④ 模型策略

```

model:
  type: knowledge_qa          # 复用 resolveOrganizeModelID 的模型类型解析
  model_id: ""                # 空 = 用租户默认模型
  temperature: 0.35
  max_tokens: 1800
  thinking: false
  timeout_seconds: 180
  retry:
    max_attempts: 2
    backoff: exponential

# ⑤ 输出契约
output:
  format: markdown            # markdown | json
  target: sprout_report       # sprout_report | output | memory_note | memory_meta
  title_template: "{{memory_title}}" 教研提炼"
  renderer: sprout_v1         # 内置渲染器枚举

# 查看契约 (D13)：决定产物是否"便于自己查看"。缺 fields 的模板不允许发布。
audience: self               # self (本期固定) | others (预留, 见 Q9)
fields:                       # 结构化字段, 前台据此生成筛选器、列表列与排序项
  - key: topic
    label: 教研主题
    type: string               # string | text | enum | date | date_range | tags | number
    source: model              # model (模型产出) | derived (系统派生) | memory (取自记忆元数据)
    filterable: true
    sortable: false
  - key: occurred_range
    label: 涉及时间
    type: date_range
    source: derived
    sortable: true
  - key: tags
    label: 标签
    type: tags
    source: model
    filterable: true
  - key: action_items
    label: 待办
    type: number
    source: derived
    filterable: true
view:

```

```

primary: card                                # 列表展示样式: list | card | timeline | grouped
group_by: tags                               # 可选, 按某字段分组
sort: occurred_range_desc
summary_field: topic                         # 列表 / 卡片上显示的主字段

json_schema:                                # format=json 时必填 (JSON Schema 子集)
  type: object
  properties:
    title: { type: string }
    summary: { type: string }
    tags: { type: array, items: { type: string }, maxItems: 6 }
  required: [title, summary, tags]
required_sections:                           # 质量门禁的章节契约
  - pattern: "^## \\d{2}\\.\\.+\"
    min_count: 3
  - pattern: "🌱 观察"
  - pattern: "✨ 提炼"
min_runes: 300
max_runes: 6000
forbidden_phrases: ["作为AI", "作为一个AI", "无法确定"]
citation: required                           # off | optional | required (required 时门禁校验 [M1] 锚点; 存量 4 条迁移)

# ⑥ 质量门禁
guardrails:
  on_violation: repair                       # repair | fallback | reject
  max_repair_attempts: 1
  repair_instruction: |
    上一次输出未满足以下要求: {{violations}}
    请严格修正后重新输出, 不要解释。

# ⑦ 降级策略
fallback:
  enabled: true
  strategy: skeleton                         # skeleton | passthrough | none
  hint: "AI 模型未配置, 已生成基础骨架, 可进入编辑完善。"
  skeleton_file: fallbacks/sprout_review.md

```

5.3 模板示例：招生增长 → 线索跟进清单

```
version: 1
template:
  key: admissions_growth
  name: 招生线索跟进清单
  scene: admissions_growth
  icon: target
  status: enabled
  visibility: platform
trigger:
  surfaces: [memory_menu, memory_bulk, workbench]
  memory_kinds: [note, record, audio]
  min_memories: 1
  max_memories: 50
input:
  select: filter
  filter: { tags: [招生, 试听, 线索], since: 14d, limit: 50 }
  ordering: occurred_at_desc
  budget: { max_memories: 50, max_runes_per_memory: 1200, max_total_runes: 9000 }
  context: [tenant.role_label, user.nickname]
instructions:
  system: |
    你是园所招生运营助手。输出中文 Markdown 表格与清单，不输出 JSON。
  task: |
    请把以下 {{memory_count}} 条招生相关记忆，整理成一份「招生线索跟进清单」。

    用户角色: {{tenant.role_label}}
    时间范围: {{memory_range}}

    输出要求:
    - "## 01. 本周线索总览": 一句话结论 + 线索条数。
    - "## 02. 待跟进线索": Markdown 表格，列固定为
      | 线索编号 | 家长/孩子称呼 | 关注点 | 当前状态 | 下一步动作 | 建议时限 |
    - "## 03. 风险与流失预警": 列出可能流失的线索与原因。
    - "## 04. 本周动作清单": 3-5 条可执行动作，标注负责人角色。

    约束:
    - 只使用记忆中出现的称呼，不得编造姓名与电话。
    - 信息不足的字段填"待补充"，不要猜测。

    记忆列表:
    {{memories}}
```

```

variables: [template.name, memory_count, memory_range, tenant.role_label, memories]
model: { type: knowledge_qa, temperature: 0.25, max_tokens: 2200, thinking: false }
output:
  format: markdown
  target: output
  title_template: "招生线索跟进清单 · {{memory_range}}"
  renderer: generic_md_v1
  audience: self
  fields:
    - { key: lead_name, label: 家长/孩子称呼, type: string, source: model, filterable: true }
    - { key: status, label: 当前状态, type: enum, source: model, filterable: true, enum: [未联系, 已联系, 已]
    - { key: next_action, label: 下一步动作, type: string, source: model }
    - { key: risk, label: 流失风险, type: enum, source: model, filterable: true, enum: [低, 中, 高] }
    - { key: occurred_range, label: 涉及时间, type: date_range, source: derived, sortable: true }
    - { key: pending_count, label: 待跟进数, type: number, source: derived, filterable: true }
  view:
    primary: list
    group_by: status
    sort: risk_desc
    summary_field: lead_name
  required_sections:
    - { pattern: "^## 01\\.\"", min_count: 1 }
    - { pattern: "^## 02\\.\"", min_count: 1 }
    - { pattern: "\\| 线索编号 \\|\"", min_count: 1 }
  min_runers: 400
  forbidden_phrases: ["作为AI", "无法确定", "张三", "李四"]
  citation: required
  guardrails: { on_violation: repair, max_repair_attempts: 1 }
  fallback: { enabled: true, strategy: passthrough, hint: "AI 不可用, 已按记忆原文汇总, 可继续编辑。" }

```

其余 7 条场景模板结构同构，差异只在 `scene / filter / fields / view / required_sections`。

注意这个例子的 `fields` 里有三个 `source: derived`（涉及时间、待跟进数）—— 它们不依赖模型产出，是这个模板即使模型跑偏也仍然可筛的稳定底座（见 §5.5）。

5.4 变量白名单与渲染规则

可渲染变量全集

变量	说明	最大长度
<code>{{template.name}}</code>	模板名	64
<code>{{memory_count}}</code>	记忆条数	—
<code>{{memory_title}}</code>	首条记忆标题	512
<code>{{memory_range}}</code>	记忆时间范围（如 <code>2026-09-15 ~ 2026-09-21</code> ）	—
<code>{{tenant.role_label}}</code>	角色中文名	32
<code>{{tenant.role_config}}</code>	角色配置文本	512
<code>{{user.nickname}}</code>	用户昵称	64
<code>{{memories}}</code>	记忆列表块（见下）	受 <code>max_total_runes</code> 约束

`{{memories}}` 的渲染格式（每条记忆追加编号，供引用与门禁校验）

[M1] 标题：中班磨课记录
类型：笔记 来源：手动输入 时间：2026-09-18 14:30
内容：
.....（受 <code>max_runes_per_memory</code> 截断，超出以「...」结尾）

渲染规则（强制）

- `variables` 未声明的变量出现 → 渲染失败，任务直接进入 `failed`，不调用模型（避免静默出错）。
- 所有变量渲染后统一做长度二次裁剪，裁剪发生在拼接前，防止注入撑爆上下文。
- 用户可控文本（昵称、记忆正文）在进入 `{{memories}}` 时，`{{` 与 `}}` 被转义为 `{ { / } }`，防止二次渲染。
- `{{memories}}` 超预算时按 `ordering` 顺序截断，并在块尾追加「（另有 N 条记忆因篇幅未纳入）」。
- 渲染结果计算 `prompt_hash = sha256(system + task_rendered)`，随产物落库。

5.5 「便于自己查看」的四条硬要求

定义里的"便于自己查看的数据"如果不能被施工，就只是一句口号。落成四条可校验的要求：

#	要求	具体口径	校验方式
V1	可筛	产物必须带 ≥2 个 <code>filterable</code> 字段； <code>enum</code> / <code>tags</code> / <code>date_range</code> 类字段必须给出取值或区间	模板发布前 Schema 校验：无字段 → 拒绝发布
V2	可扫	列表/卡片上有一眼能懂的主字段（ <code>summary_field</code> ），且 <code>max_runes</code> 有上限；正文不是唯一取用入口	前台走查：不看正文能否判断"这条是不是我要找的"
V3	可跳回	结论带 <code>[Mx]</code> 锚点，点一下回到原始记忆；锚点失效（记忆已删）时降级为不可点但保留编号	<code>citation: required</code> 时门禁校验锚点存在且编号在输入范围内
V4	可复用	产物能被再次选中作为整理输入，也能另存为成果；不产生"只能看一次"的死内容	手工验收：产物可被勾选再次整理

V1 是硬门槛，V3 是新场景模板的硬门槛。存量 4 条迁移链路例外（D14）：迁移优先保证行为等价，其 `citation` 保持 `optional`，V1 的字段由系统派生补齐而非改提示词。

字段从哪来（三种 source）

source	含义	例子	对模型的要求
model	由模型在正文中产出，系统抽取	主题、标签、结论	提示词里必须显式要求，且 <code>json_schema</code> 或抽取规则可解析
derived	系统从输入/产物派生，不依赖模型	涉及时间、记忆条数、待办条数	无需模型配合，优先用这类兜住 V1
memory	直接取自输入记忆的元数据	类型、来源、所属班级	无需模型配合

施工建议：**能派生就不要让模型产出**。模型产出的字段越多，越容易缺项、越难校验；`derived` 字段是"可筛"能力的稳定底座。

6. 数据模型设计

6.1 表清单

表	性质	说明
<code>organize_templates</code>	新增	模板主体。平台级唯一真源， <code>spec</code> 存的是编辑态草稿
<code>organize_template_versions</code>	新增	已发布版本快照。运行期只读这里
<code>organize_configs</code>	新增	用户级「整理配置」（对应前台工作台「我的整理」卡片）：名称 / 指令 / 专家 / 执行周期 / 模板引用。模板是平台资产，配置是用户实例 —— 对齐服务页「服务实例 ← 服务模板」的关系
<code>organize_jobs</code>	新增	整理任务与执行状态。含落定后的要求快照（ <code>requirement</code> ）与提供方（ <code>requirement_provider</code> ）；新增 <code>config_id</code> 指向触发它的整理配置
<code>organize_memories</code>	变更	<code>metadata</code> 增加模板溯源键（无 DDL 变更）
<code>organize_sprout_reports</code>	变更	新增 <code>template_key</code> 、 <code>template_version</code> 、 <code>fields</code>
<code>organize_outputs</code>	变更	新增 <code>template_key</code> 、 <code>template_version</code> 、 <code>job_id</code> 、 <code>fields</code> 、 <code>citations</code>

6.2 新增表 DDL (`migrations/versioned/000101_organize_templates.up.sql`)

```
-- Migration: 000101_organize_templates
-- Introduces configurable organize templates and the job pipeline.
-- Runtime reads the published snapshot in organize_template_versions,
-- never organize_templates.spec (which holds the in-edit draft).

CREATE TABLE IF NOT EXISTS organize_templates (
  id                VARCHAR(36) PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id         BIGINT NOT NULL DEFAULT 0,          -- 0 = 平台级 (本期固定 0)
  tkey              VARCHAR(64) NOT NULL,
  name              VARCHAR(128) NOT NULL,
  scene             VARCHAR(64) NOT NULL DEFAULT '',
  icon              VARCHAR(64) NOT NULL DEFAULT '',
  description        VARCHAR(512) NOT NULL DEFAULT '',
  visibility         VARCHAR(16) NOT NULL DEFAULT 'platform', -- 本期固定 platform
  status            VARCHAR(16) NOT NULL DEFAULT 'draft',    -- draft | enabled | disabled
  sort              INTEGER NOT NULL DEFAULT 100,
  spec              JSONB NOT NULL DEFAULT '{}':::jsonb,    -- 编辑态草稿 (§5.2 的结构)
  published_version  INTEGER NOT NULL DEFAULT 0,            -- 指向生效版本; 0 = 从未发布
  is_seed           BOOLEAN NOT NULL DEFAULT FALSE,        -- 是否由初始化种子写入
  created_by        VARCHAR(36) NOT NULL DEFAULT '',
  updated_by        VARCHAR(36) NOT NULL DEFAULT '',
  published_by      VARCHAR(36) NOT NULL DEFAULT '',
  published_at      TIMESTAMP WITH TIME ZONE,
  created_at        TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at        TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  deleted_at        TIMESTAMP WITH TIME ZONE,
  CONSTRAINT uq_organize_templates_key UNIQUE (visibility, tkey),
  CONSTRAINT chk_organize_templates_visibility
    CHECK (visibility IN ('platform', 'tenant', 'personal')),
  CONSTRAINT chk_organize_templates_status
    CHECK (status IN ('draft', 'enabled', 'disabled')),
  -- 不变量: 只有已发布的模板才允许启用
  CONSTRAINT chk_organize_templates_published
    CHECK (status <> 'enabled' OR published_version > 0)
);

CREATE INDEX IF NOT EXISTS idx_organize_templates_scene
  ON organize_templates(scene, sort)
  WHERE deleted_at IS NULL;

CREATE TABLE IF NOT EXISTS organize_template_versions (
```

```

id            VARCHAR(36) PRIMARY KEY DEFAULT uuid_generate_v4(),
template_id   VARCHAR(36) NOT NULL,
tenant_id     BIGINT NOT NULL DEFAULT 0,
tkey          VARCHAR(64) NOT NULL,
version       INTEGER NOT NULL,
spec          JSONB NOT NULL DEFAULT '{}'::jsonb,
change_note   VARCHAR(512) NOT NULL DEFAULT '',
created_by    VARCHAR(36) NOT NULL DEFAULT '',
created_at    TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT uq_organize_template_versions UNIQUE (template_id, version)
);

```

```

CREATE INDEX IF NOT EXISTS idx_organize_template_versions_key
ON organize_template_versions(tkey, version DESC);

```

```

CREATE TABLE IF NOT EXISTS organize_jobs (
id            VARCHAR(36) PRIMARY KEY DEFAULT uuid_generate_v4(),
tenant_id     BIGINT NOT NULL,
user_id       VARCHAR(36) NOT NULL,
requirement_provider VARCHAR(16) NOT NULL DEFAULT 'preset', -- preset (本期) | custom (预留)
requirement   JSONB NOT NULL DEFAULT '{}'::jsonb,          -- 落地后的要求快照, 创建后不可变 (D11)
template_key  VARCHAR(64) NOT NULL DEFAULT '',              -- provider=custom 时为空
template_version INTEGER NOT NULL DEFAULT 0,
spec_source   VARCHAR(16) NOT NULL DEFAULT 'platform', -- platform (tenant / personal 预留)
target        VARCHAR(32) NOT NULL DEFAULT '',            -- sprout_report | output | ...
target_id     VARCHAR(36) NOT NULL DEFAULT '',
status        VARCHAR(24) NOT NULL DEFAULT 'queued',
stage         VARCHAR(24) NOT NULL DEFAULT '',
progress      INTEGER NOT NULL DEFAULT 0,
memory_ids    JSONB NOT NULL DEFAULT '[]'::jsonb,
memory_count  INTEGER NOT NULL DEFAULT 0,
model_id      VARCHAR(64) NOT NULL DEFAULT '',
prompt_hash   VARCHAR(64) NOT NULL DEFAULT '',
violation     VARCHAR(512) NOT NULL DEFAULT '',            -- 门禁未通过原因
repair_count  INTEGER NOT NULL DEFAULT 0,
error_message VARCHAR(512) NOT NULL DEFAULT '',
input_runes   INTEGER NOT NULL DEFAULT 0,
output_runes  INTEGER NOT NULL DEFAULT 0,
duration_ms   INTEGER NOT NULL DEFAULT 0,
metadata      JSONB NOT NULL DEFAULT '{}'::jsonb,
started_at    TIMESTAMP WITH TIME ZONE,
finished_at   TIMESTAMP WITH TIME ZONE,
created_at    TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

```

```

updated_at      TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT chk_organize_jobs_status
    CHECK (status IN ('queued','running','repairing','completed','fallback','failed','canceled')),
CONSTRAINT chk_organize_jobs_requirement_provider
    CHECK (requirement_provider IN ('preset','custom')),
CONSTRAINT chk_organize_jobs_progress
    CHECK (progress BETWEEN 0 AND 100)
);

CREATE INDEX IF NOT EXISTS idx_organize_jobs_scope_time
    ON organize_jobs(tenant_id, user_id, created_at DESC);
CREATE INDEX IF NOT EXISTS idx_organize_jobs_status
    ON organize_jobs(status) WHERE status IN ('queued','running','repairing');
CREATE INDEX IF NOT EXISTS idx_organize_jobs_template
    ON organize_jobs(template_key, created_at DESC);

```

6.3 存量表变更

```

-- organize_sprout_reports: 模板溯源 + 查看契约
ALTER TABLE organize_sprout_reports
    ADD COLUMN IF NOT EXISTS template_key VARCHAR(64) NOT NULL DEFAULT '',
    ADD COLUMN IF NOT EXISTS template_version INTEGER NOT NULL DEFAULT 0,
    ADD COLUMN IF NOT EXISTS fields JSONB NOT NULL DEFAULT '{} '::jsonb;    -- 结构化字段 (V1)

-- organize_outputs: 模板溯源 + 任务关联 + 查看契约
ALTER TABLE organize_outputs
    ADD COLUMN IF NOT EXISTS template_key VARCHAR(64) NOT NULL DEFAULT '',
    ADD COLUMN IF NOT EXISTS template_version INTEGER NOT NULL DEFAULT 0,
    ADD COLUMN IF NOT EXISTS job_id VARCHAR(36) NOT NULL DEFAULT '',
    ADD COLUMN IF NOT EXISTS fields JSONB NOT NULL DEFAULT '{} '::jsonb,    -- 结构化字段 (V1)
    ADD COLUMN IF NOT EXISTS citations JSONB NOT NULL DEFAULT '[] '::jsonb; -- [{"ref":"M1","memory_id":'

-- organize_memories: 批量整理需要按 tag / scene 检索, 加表达式索引
CREATE INDEX IF NOT EXISTS idx_organize_memories_tags
    ON organize_memories USING GIN ((metadata -> 'tags'))
    WHERE deleted_at IS NULL;

-- 查看契约 (V1) 的检索前提: 产物按字段筛选
CREATE INDEX IF NOT EXISTS idx_organize_outputs_fields
    ON organize_outputs USING GIN (fields)
    WHERE deleted_at IS NULL;

```

迁移策略：全部为 `ADD COLUMN ... DEFAULT` 与新建索引，**不重建表、不回填存量数据**。存量产物的 `template_key` 留空、`fields` 为空对象，前端按"历史内容"展示 —— 历史产物不满足 V1，这是可接受的：**V1 是发布门槛，不是历史数据的整改要求**。

需同步产出 `mysql/` 与 `sqlite/` 方言版本（项目已有的三方方言目录约定）。

6.4 索引设计要点

- `organize_jobs` 的活跃态索引用**部分索引**（`WHERE status IN (...)`），避免任务表膨胀后拖慢状态轮询。
- `organize_templates` 的唯一键是 `(visibility, tkey)`：本期 `visibility` 恒为 `platform`，等价于 key 全局唯一；同时给未来的 `tenant` / `personal` 层预留命名空间，将来不需要再改约束。
- `chk_organize_templates_published` 把"只有发布过才能启用"这条业务不变量下沉到数据库 —— 应用层就算有 bug 也绕不过去。
- `metadata -> 'tags'` 的 GIN 索引是批量取数的性能前提，未命中场景退化为顺序扫描 + LIMIT。

7. 执行管线

7.1 六阶段



① 是 D11 的落地点：用户表达要求（选方案 + 定范围）与系统落定要求（校验合法性、冻结为不可变快照）都在这一阶段完成，**只有落定成功才进入 ②，且后续阶段不再回头看模板的最新状态**。

阶段	输入	输出	失败处置
① 落定要求	要求提供方（本期 = <code>template_key</code> ）+ 租户/用户上下文 + 用户指定的范围	要求快照（目标产物 / 口径结构 / 作用范围 / 红线）+ 模板版本号	模板不存在/已停用 → <code>failed</code> ，提示"该整理方案已下线"；范围越界 → 提示并回退到模板上限
② 装配输入	快照中的作用范围（记忆 ID 或 filter 条件）	记忆列表 + 预算裁剪 + 输入字数	无有效记忆 → <code>failed</code> ，提示"没有可整理的记忆"
③ 渲染指令	快照 + 记忆列表 + 上下文	<code>system</code> / <code>task</code> 文本 + <code>prompt_hash</code>	未声明变量 → <code>failed</code> （不调模型）
④ 调用模型	渲染结果 + 模型策略	原始输出文本	超时/报错 → 重试 → <code>fallback</code>
⑤ 契约校验	原始输出 + 快照中的口径结构 + 查看契约（字段/锚点）	校验通过内容 + 抽取与派生的字段集 / 违规清单	违规 → 自修复 1 次 → 仍违规 → <code>fallback</code>
⑥ 落库/降级	通过内容 + 字段集，或骨架	产物（正文 + 字段 + 锚点）+ <code>job</code> 状态更新	落库失败 → 重试 → <code>failed</code> ，保留原始输出到 <code>job.metadata</code>

7.2 整理任务状态机



状态	含义	前端文案
<code>queued</code>	已入队	整理任务已创建，排队中
<code>running</code>	模型生成中	AI 正在整理你的记忆...
<code>repairing</code>	输出不合格，自修复中	正在按内容结构要求重新整理
<code>completed</code>	通过全部契约	整理完成
<code>fallback</code>	已降级为骨架/原文	已生成基础版，可继续编辑完善
<code>failed</code>	不可恢复失败	整理失败：{原因}，可重试
<code>canceled</code>	用户取消	已取消

流转规则

- R1: 仅 `queued` 可被取消；`running` 取消仅标记，不强杀模型调用。
- R2: `repairing` 最多一次（`max_repair_attempts: 1`），超出转 `fallback`。
- R3: `fallback` 与 `failed` 均可重试，重试生成新 job，不覆盖旧 job 记录。
- R4: 同一用户对同一批记忆 + 同一模板，5 分钟内重复提交返回既有 job（幂等键 = `tenant + user + hash(template_key, sorted(memory_ids))`）。

7.3 幂等、重试、降级

机制	设计
幂等	幂等键落 <code>organize_jobs.metadata.idempotency_key</code> ，5 分钟窗口内复用
队列	复用 <code>async</code> ，新增任务类型 <code>TypeOrganizeJobRun</code> ，队列走 <code>types.QueueForTaskType</code>
重试	模型层重试（ <code>max_attempts: 2</code> ，指数退避）；队列层 <code>MaxRetry(2)</code> 、 <code>Timeout(180s)</code>
降级	三级： <code>skeleton</code> （模板骨架）→ <code>passthrough</code> （记忆原文拼接）→ <code>none</code> （不产出，仅报错）
可观测	每个阶段写 <code>organize_jobs.stage</code> ，SSE 推送进度（复用 <code>agent-run events</code> 的推送模式）

7.4 质量门禁与自修复

校验项（按 `output` 契约逐条执行）

校验项	判定	不通过文案
<code>format</code>	JSON 可解析 / Markdown 非空	输出格式不符合要求
<code>json_schema</code>	必填字段齐全、类型正确、数组不超上限	输出缺少必填字段：tags
<code>required_sections</code>	正则命中次数 ≥ <code>min_count</code>	缺少内容结构： <code>## 01.</code> 至少 3 处
<code>min_runes</code> / <code>max_runes</code>	字符数区间	内容过短，未达到 300 字
<code>forbidden_phrases</code>	不含禁用词	输出包含禁用表述
<code>citation: required</code>	至少出现一次 <code>[M\d+]</code> 引用，且编号落在输入记忆范围内	结论未标注来源记忆
<code>fields</code> 完备性 (V1)	每个 <code>source=model</code> 的字段都能从输出中抽到值且非空； <code>derived</code> 字段由系统补齐	输出缺少信息项：标签
锚点有效性 (V3)	每个 <code>[Mx]</code> 都能对应到一条输入记忆	引用了不存在的来源记忆

自修复流程

1. 收集全部违规项，渲染 `repair_instruction`（模板中可配置）。
2. 追加一轮对话：原输出 + 违规清单 + 修正要求。
3. 再校验一次；通过则 `completed`，仍违规则按 `on_violation` 处置（默认 `fallback`）。

4. 全程 `violation` 与 `repair_count` 落库，供模板效果看板分析。

该设计直接补上现状缺口 3： `organize_sprout.go` 目前对模型输出零校验。

7.5 模板加载、缓存与失效

模板是"高频读、极低频写"的数据，且运行期只读已发布快照，所以：

环节	做法
初始化种子	启动时若 <code>organize.template_seed.enabled: true</code> ，读 <code>seed/*.yaml</code> ，仅插入库中不存在的 key
启动自检	对库里所有 <code>status = 'enabled'</code> 的模板跑一遍 Schema 校验；不合法的自动置为 <code>disabled</code> 并告警，不阻塞启动
读路径	进程内缓存（key = <code>tkey</code> ，value = 已发布 spec + version + prompt 模板），命中即用
失效	后台发布 / 停用 / 删除时主动失效本实例缓存；多实例之间用 30s TTL 兜底
兜底	缓存与库都取不到 → 回退到代码内置基线（D8），并记 <code>job.stage = template_fallback</code>

为什么必须"启动自检 + 自动停用"：真源进了数据库，就意味着一次错误的后台编辑有可能让服务起不来、或者在运行期 panic。把校验放在启动期、把坏模板就地停用，是"拿数据库当配置源"必须付的对价。

为什么不做实时推送：模板变更频率是"人操作的频率"（一天几次），30s TTL 带来的最坏延迟完全可接受，不值得为它引入 Redis 发布订阅这样的额外依赖。

8. 功能需求（EARS）

8.1 用户发起整理

- FR-1 当用户在某条记忆的操作菜单中选择「用模板整理」时，系统应展示该记忆类型下所有可用模板，并按 `sort` 与场景相关性排序。
- FR-2 当用户在记忆列表多选 ≥ 1 条记忆并点击「整理」时，系统应只展示 `min_memories ≤ 选中数 ≤ max_memories` 的模板。
- FR-3 当用户确认模板并提交时，系统应在 1 秒内返回任务创建结果，并展示整理任务进度。
- FR-4 当整理任务完成时，系统应将产物落入手机端/PC 端对应页签（发芽报告 / 成果），并记录模板溯源信息。
- FR-5 当整理任务降级时，系统应展示模板配置的降级提示文案，并允许用户一键重试。

8.2 要求的表达与模板选择

- FR-6 当用户打开模板选择器时，系统应按 8 个经营场景分组展示模板卡片（名称、图标、一句话说明、预计产出形态），卡片文案描述产出什么而非提示词是什么。
- FR-7 当用户选中的记忆被打上 `tags` 时，系统应把命中标签的模板置顶推荐。

- **FR-8** 当用户打开模板选择器时，系统应只展示 `status = enabled` 且已有发布版本的模板；草稿、从未发布、已停用的模板不得出现在任何前台入口。
- **FR-8a** 当用户提交一次整理时，系统应把该次整理的**要求**落定为不可变快照（目标产物 / 口径结构 / 作用范围 / 红线）并随 job 冻结存储，后续任何模板改动不得改变已创建 job 的行为。
- **FR-8b** 当模板声明的 `input` 允许调整范围时，系统应允许用户在提交前收窄作用范围（时间区间、条数、标签），但不得放宽模板声明的上限（如 `max_memories`）。
- **FR-8c** 当要求的提供方为 `custom` 时（本期未启用），系统应在界面上提供自由描述入口并把描述交由要求解析器落定；未启用时该入口不得出现。

8.2a 整理配置（工作台「我的整理」）

- **FR-8d** 当用户在工作台点击「新建整理」或某条整理模板时，系统应提供配置表单，字段为：**名称（必填）**、**指令（可从模板预填）**、**专家（可选，多选）**、**执行周期（手动 / 每天 / 每周一 / 每月 1 日）**；字段与交互对齐服务创建弹窗 `ServiceCreateDialog.vue`。
- **FR-8e** 当用户保存整理配置时，系统应创建一条 `organize_configs` 记录并使其立即出现在「我的整理」卡片网格；保存时**不触发执行**。
- **FR-8f** 当整理配置的执行周期非「手动」时，系统应由调度器按周期对该配置**新产生的记忆**自动创建整理任务，任务记入该配置的任务列表；周期触发创建的任务与手动发起的任务走同一条落定管线（D11），仅 `trigger` 来源标记不同。
- **FR-8g** 当用户编辑整理配置（「设置」）时，系统应回填配置表单；配置变更**只影响之后创建的任务**，不改变已有任务及其产出（落定快照不可变）。
- **FR-8h** 当整理配置引用的模板被停用或下线时，系统应保留该配置与历史任务，周期触发改为跳过并给出提示；用户可在「设置」里改选其他模板。

8.3 平台管理员维护模板

- **FR-9** 当平台管理员在后台新建模板时，系统应提供字段级表单（基础信息、触发条件、输入装配、指令、模型参数、结构要求、降级策略）并即时校验 Schema。
- **FR-10** 当平台管理员点击「试跑」时，系统应支持选择 1-5 条真实记忆执行一次**不落库**的整理，展示渲染后的指令全文、模型原始输出、门禁校验明细、耗时与 token。
- **FR-11** 当平台管理员保存为草稿时，系统应允许保存不完整配置（Schema 校验降级为警告）。
- **FR-12** 当平台管理员发布模板时，系统应要求填写变更说明，写入版本快照、递增 `published_version` 并置为 `enabled`。
- **FR-13** 当平台管理员回滚版本时，系统应以指定历史版本内容新建一个发布版本，历史版本记录不删除。
- **FR-14** 仅平台系统管理员可创建、编辑、发布、停用、删除整理模板；其他角色调用写接口应返回 403。
- **FR-15** 当平台管理员停用模板时，系统应使该模板立即从所有前台选择器中消失，但保留其历史产物不变。

8.4 结果编辑与反馈

- **FR-16** 当用户编辑 AI 整理产物后保存时，系统应记录「编辑距离」（用于模板效果看板的采纳率指标）。
- **FR-17** 当用户对产物点击「有帮助 / 没帮助」时，系统应记录到 job 的反馈字段。

8.5 产物的查看（"便于自己查看"的验收口径）

- FR-18 当整理任务完成时，系统应把产物以可筛选的数据视图呈现（列表 / 卡片 / 时间轴，按模板 `view` 配置），并展示 `summary_field` 与声明字段；不得只有一篇需要通读的正文。
- FR-19 当产物含 `filterable` 字段时，系统应提供对应的筛选与排序控件，且筛选状态可保留。
- FR-20 当用户点击产物中的 `[Mx]` 锚点时，系统应跳转到对应的原始记忆；锚点对应记忆已删除时，应降级为不可点击并保留编号，不得报错。
- FR-21 当用户勾选已有产物再次发起整理时，系统应允许把产物作为输入（`input.select: outputs`），支撑"二次沉淀"。
- FR-22 当模板未声明 `fields` 时，系统应拒绝发布并提示"至少需要一个可筛选字段"（V1 门槛）。
- FR-23 当模板 `citation: required` 而输出未带锚点时，系统应判定违规并进入自修复流程（复用 §7.4）。

9. API 契约

9.1 模板

运行期读接口（前台消费）

方法	路径	权限	说明
GET	<code>/organize/templates</code>	Viewer	列出可用模板（仅 <code>enabled</code> 的已发布版本），含最终配置摘要。查询参数： <code>scene</code> 、 <code>surface</code> 、 <code>memory_ids</code> （用于筛出可用模板）
GET	<code>/organize/templates/scenes</code>	Viewer	经营场景列表（替代前端硬编码的 <code>DISCOVER_CATEGORIES</code> ）
GET	<code>/organize/templates/:key</code>	Viewer	模板详情（含 <code>instructions</code> 、 <code>output</code> 契约）

后台管理接口（平台系统管理员）

权限用既有的 `g.SystemAdmin()` 守卫（对应 `User.IsSystemAdmin`，用法对齐 `router.go` 中模型管理与账单管理），与 `admin` 前端的 `requiresSystemAdmin` 一一对应。接口挂在 `/organize/templates` 前缀下、由守卫区分，不另开 `/admin` 前缀——与 `admin/src` 通过 `@ → ../frontend/src` 复用 API 层的既有做法保持一致。

方法	路径	权限	说明
GET	<code>/organize/templates/manage</code>	SystemAdmin	管理列表：含草稿态、停用态、发布人、发布时间
POST	<code>/organize/templates</code>	SystemAdmin	新建（默认 <code>draft</code> ）
PUT	<code>/organize/templates/:key</code>	SystemAdmin	保存草稿（带 <code>version</code> 乐观锁）
DELETE	<code>/organize/templates/:key</code>	SystemAdmin	软删除
POST	<code>/organize/templates/:key/publish</code>	SystemAdmin	发布：写版本快照、递增 <code>published_version</code> 、置 <code>enabled</code> 。 <code>change_note</code> 必填
POST	<code>/organize/templates/:key/disable</code>	SystemAdmin	停用（不影响历史产物）
POST	<code>/organize/templates/:key/rollback</code>	SystemAdmin	回滚：以指定 <code>version</code> 作为新的发布版本
GET	<code>/organize/templates/:key/versions</code>	SystemAdmin	版本列表 + 版本间 diff
POST	<code>/organize/templates/:key/preview</code>	SystemAdmin	试跑：返回渲染后指令、模型原始输出、门禁明细、耗时与 token（不落库）
POST	<code>/organize/templates/import</code>	SystemAdmin	从 YAML 导入（冲突策略：跳过 / 覆盖草稿 / 覆盖并发布）
GET	<code>/organize/templates/export</code>	SystemAdmin	导出 YAML（跨环境同步用）
GET	<code>/organize/templates/metrics</code>	SystemAdmin	模板与版本维度的效果指标

9.2 整理任务

方法	路径	权限	说明
POST	<code>/organize/jobs</code>	Viewer	创建整理任务。Body: <code>{requirement_provider: "preset", template_key, scope: {memory_ids[] \ filter{tags,from,to}, max_memories?}, model_id?}</code> ；服务端落定为要求快照后返回 job
GET	<code>/organize/jobs/:id</code>	Viewer	任务详情（状态、进度、产物指针、门禁结果、 落定后的要求快照 ）
GET	<code>/organize/jobs/:id/events</code>	Viewer	SSE 进度流（复用 agent-run events 模式）
POST	<code>/organize/jobs/:id/retry</code>	Viewer	重试（生成新 job）
POST	<code>/organize/jobs/:id/cancel</code>	Viewer	取消
GET	<code>/organize/jobs</code>	Viewer	任务列表（筛选 <code>status</code> 、 <code>template_key</code> ）

9.2.1 产物查看

「便于自己查看」的接口支撑（FR-18 ~ FR-21）：

方法	路径	权限	说明
GET	<code>/organize/outputs</code>	Viewer	产物列表。查询参数: <code>target</code> 、 <code>template_key</code> 、 <code>q</code> (全文)、 <code>fields[标签]=...</code> 、 <code>sort</code> 、 <code>cursor</code> ; 返回项含 <code>fields</code> 与 <code>view</code> 元信息
GET	<code>/organize/outputs/facets</code>	Viewer	当前筛选条件下的可用筛选项与计数 (标签云、时间直方图), 避免前端全量拉数
GET	<code>/organize/outputs/:id</code>	Viewer	产物详情: 正文 + 字段 + <code>citations</code> (含每条锚点对应的记忆标题, 供跳转前预览)
GET	<code>/organize/outputs/:id/citations/:ref</code>	Viewer	按锚点解析原始记忆 (<code>[M1]</code> → <code>memory_id</code>) ; 记忆已删返回 200 + <code>missing: true</code> , 由前端降级展示

9.2.2 整理配置 (对应前台「我的整理」卡片)

方法	路径	权限	说明
GET	<code>/organize/configs</code>	Viewer	配置列表。查询参数: <code>q</code> 、 <code>sort</code> (<code>updated_at</code> <code>todo</code>)、 <code>cursor</code> ; 返回项含模板名、执行周期、专家、近期任务计数与状态
POST	<code>/organize/configs</code>	Viewer	新建配置。Body: <code>{name, instruction, template_key?, expert_ids[], schedule}</code> ; 创建不触发执行 (FR-8e), 返回体含 <code>next_run_at</code>
GET	<code>/organize/configs/:id</code>	Viewer	配置详情 (含指令、专家、周期、绑定的模板与版本)
PUT	<code>/organize/configs/:id</code>	Viewer	更新配置。仅影响之后创建的任务 (FR-8f) ; 已存在任务的 <code>requirement</code> 快照不变
DELETE	<code>/organize/configs/:id</code>	Viewer	删除配置 (软删除)。其历史任务与产物保留
POST	<code>/organize/configs/:id/run</code>	Viewer	手动发起一次整理 → 返回新建的 job (等价于 §9.2 的 <code>POST /organize/jobs</code> 但由配置提供要求与范围默认值)
GET	<code>/organize/configs/:id/jobs</code>	Viewer	该配置的任务列表 (时间线数据源)。返回任务状态 / 时间 / 范围 / 模板版本 / 产出指针, 按时间倒序

9.3 存量接口兼容

存量接口	处置
POST /organize/sprout-reports/from-memory	保留，内部改为"以 sprout_review 平台模板创建 job"的语法糖，响应结构不变
POST /organize/memories/upload	内部走 note/audio_transcribe 模板； transcription_status 语义不变
POST /organize/outputs/upload	内部走 output/card_meta 模板
GET /organize/discover	分类来源改为模板 scene 聚合；响应结构不变

9.4 错误码

码	HTTP	含义
ORGANIZE_TEMPLATE_NOT_FOUND	404	模板不存在或已下线
ORGANIZE_TEMPLATE_DISABLED	409	模板已停用
ORGANIZE_TEMPLATE_NOT_PUBLISHED	409	模板存在但从未发布，不可用于整理
ORGANIZE_TEMPLATE_INVALID_SPEC	400	模板配置校验失败（附字段级错误）
ORGANIZE_TEMPLATE_VERSION_CONFLICT	409	乐观锁冲突，需刷新后重试
ORGANIZE_TEMPLATE_CHANGE_NOTE_REQUIRED	400	发布时未填写变更说明
ORGANIZE_JOB_NO_MEMORY	400	无有效记忆可整理
ORGANIZE_JOB_TOO_MANY_MEMORY	400	超出模板 max_memories
ORGANIZE_JOB_RENDER_FAILED	500	指令渲染失败（未声明变量）
ORGANIZE_JOB_MODEL_UNAVAILABLE	503	模型不可用，已降级
ORGANIZE_JOB_GUARDRAIL_REJECTED	422	输出未通过内容结构要求

10. 界面与交互

10.1 页面与入口

前台（ frontend/ ）

页面 / 组件	变更
<code>views/organize/OrganizeWorkspace.vue</code>	顶部新增「整理」入口按钮
<code>views/organize/components/OrganizeTemplatePicker.vue</code>	新增：模板选择弹层（场景分组 + 卡片）
<code>views/organize/OrganizeWorkbench.vue</code>	新增：整理工作台（hub 首页，信息架构对齐 ServiceHub 列表页 —— 上「我的整理」实例卡片 / 下「整理模板」，见 §10.3）
<code>views/organize/components/OrganizeJobProgress.vue</code>	新增：整理进度（SSE）
<code>views/organize/components/OrganizeOutputView.vue</code>	新增：产物查看视图（按 <code>view</code> 渲染列表 / 卡片 / 时间轴 + 字段筛选器 + 锚点跳转），是"便于自己查看"的落点
<code>views/organize/components/OrganizeConfigDialog.vue</code>	新增：新建 / 编辑整理配置弹层（名称 / 指令 / 专家 / 执行周期），字段与交互对齐 <code>views/service/ServiceCreateDialog.vue</code>
<code>views/organize/OrganizeConfigDetail.vue</code>	新增：整理详情二级页 —— 任务时间线（轴点 + 任务卡）、「设置」「+ 再次整理」
<code>views/organize/organizeRoutes.ts</code>	变更：页签常量新增 <code>hub</code> / <code>mine</code> ， <code>resolveOrganizeRoutePath</code> 兜底改为 <code>hub</code> （详见 §10.6）
<code>views/organize/discoverCategories.ts</code>	改为从 <code>/organize/templates/scenes</code> 拉取，保留本地兜底常量
<code>api/organize/index.ts</code>	新增模板与整理任务相关类型与请求函数（后台复用同一份）

平台后台（`admin/`）

文件	变更
<code>admin/src/views/AdminOrganizeTemplates.vue</code>	新增：模板列表页
<code>admin/src/views/AdminOrganizeTemplateEditor.vue</code>	新增：模板编辑器 + 试跑面板
<code>admin/src/views/AdminOrganizeTemplateVersions.vue</code>	新增：版本列表 / diff / 回滚
<code>admin/src/config/navigation.ts</code>	「资产治理」新增导航项「整理模板」（ <code>requiresSystemAdmin</code> ）

`admin/src` 的 `@` 别名指向 `../frontend/src`（见 `admin/vite.config.ts:116`），所以后台可以直接复用前台已有的组件与 API 层，不需要复制一份。

10.2 表达要求的弹层（选方案 + 定范围）

界面上不出现"模板""提示词"这类词 —— 用户看到的是"你想整理成什么"。

你想整理成什么?		已选 3 条记忆	
🌱 教师成长 / 教研			
教研提炼		磨课复盘	
产出提炼稿		产出复盘	
		产出观察记	
🎯 招生增长			
线索跟进清单		渠道效果周报	
范围: 近 7 天 ▼		最多 20 条 ▼	
		[取消] [开始整理]	

交互规则

- 卡片展示：名称 + 一句话说明 + **产出形态**标签 + 预计耗时。文案一律描述"产出什么"，不描述"怎么生成"。
- 命中所选记忆 **tags** 的场景置顶，未命中场景折叠。
- 选中数不满足模板 **min/max** 时卡片置灰并提示原因。
- 范围区只允许**收窄**不允许放宽（FR-8b）：上限由方案决定，用户可在此之下调时间、调条数。
- 提交前回显一句"系统将按什么整理"，让用户确认要求已被正确理解 —— 这是 D11 在界面上的可见性。
- 无任何可用方案时，展示空态："当前没有匹配的记忆类型，可联系管理员配置整理方案。"

10.3 整理工作台

信息架构对齐 **views/service/ServiceHub.vue** 的列表页（hub）模式：页头（标题 + 副标题 + 装饰图形）→ 主按钮「新建整理」→ 进行中提示条 → 上：「我的整理」整理配置卡片网格 → 下：「整理模板」模板卡片网格。4 列网格、卡片样式（品牌色图标 + 标题 + 标签行 + 两行描述 + meta 行）、排序与搜索的交互全部复用服务页的既有模式。

核心语义：工作台上的卡片是「整理配置」（**organize_configs**），不是单次产物 —— 与服务页「我的服务 = 服务实例 / 从模板创建 = 服务模板」完全同构。模板是平台资产，配置是用户按模板（或空白）创建的实例；每次执行（手动或周期触发）在配置下记为一条任务。

「新建整理」配置表单（字段对齐 **views/service/ServiceCreateDialog.vue**）

字段	形态	说明
名称	单行输入	必填；从模板进入时默认「{模板名}整理」
指令	多行文本 + 字段头「选择模板」下拉	下拉选定模板后 预填默认指令 ；产出结构（可筛字段、引用锚点）由模板决定，指令只负责「按什么口径整理」
专家	可折叠多选（可选）	复用服务创建弹窗的 option–section 交互；专家参与口径把关
执行周期	单选：手动 / 每天 / 每周一 / 每月 1 日	选了周期后由调度器按新产生的记忆自动创建任务；任务记录进入该配置的任务列表

上段：「我的整理」（已配置好的整理）

卡片元素	内容	说明
图标 + 标题	配置名称	进行中的任务同样成卡，带阶段标签（如「④ 调用模型」）与进度条
标签行	模板名 + 执行周期 + 状态	状态：进行中 / 已完成 / 待首次执行 / 已降级；历史产物标注「历史内容」
说明行	指令摘要	卡片上的描述就是这条配置的指令，所见即所配
meta 行	专家 + 计数	如「教研专家 · 2 次整理」

- 工具行：排序 + 搜索。空态引导："点「新建整理」配置一条，或从下面的整理模板开始。"
- 点击卡片 → 进入该整理的任务列表页（二级页，头部带「返回工作台」「设置」（回填配置表单）「+ 再次整理」）：每次发起整理记为一条任务，以**时间线呈现**——自上而下由新到旧，每条任务挂在轴上自己的时间点（轴：日期 + 时间 + 状态圆点；圆点配色：进行中呼吸态 / 已完成品牌色 / 刚生成绿色 / 已取代灰色）。任务卡展示状态 / 范围 / 模板版本 / 产出摘要，「查看产出」打开 §10.5 产物查看视图。重新整理是**新增一条任务**而不改写历史；被新任务取代的旧任务灰显但保留——对应 §7.1 落定快照的不可变语义。进行中的任务卡显示阶段标签与进度条，完成后产出挂到该任务下。

下段：「整理模板」

- 卡片：图标 + 模板名 + 一句话说明 + 「产出 XX」标签 + 场景短名。只展示 `status = enabled` 且已有发布版本的模板（同 FR-8）。
- 点击模板卡片 → 打开「**新建整理**」配置表单并按该模板预填（名称默认值 + 指令默认值），用户补齐专家与执行周期后保存为一条整理配置。
- 与 §10.2 的关系：§10.2 是"先选记忆 → 再表达要求"的一次性整理；§10.3 是"先配好，之后反复执行"的持久配置。两条路径的执行都汇合到同一个落定步骤，落定后行为一致（D11）。

前台术语：界面用「整理模板」「模板」是**允许的**——服务页已使用"从模板创建"的语汇（AC-16 相应放宽，仍禁止出现"提示词""参数"等实现词；「指令」一词对齐服务创建弹窗的既有用法）。

10.4 平台后台：整理模板管理（admin）

入口与权限

```
// admin/src/config/navigation.ts → 「资产治理」分组内新增
{
  key: 'organize-templates',
  label: '整理模板',
  description: '统一维护平台记忆整理模板',
  icon: 'setting',
  path: '/organize/templates',
  requiresSystemAdmin: true,    // 与「智能体」同级：仅平台系统管理员可见
}
```

与「智能体」条目完全同构 —— 那一条的描述就是"统一维护平台内置智能体"。整理模板是同一类东西：**平台治理的内容资产**。

页面一：模板列表 `/organize/templates`

- 按「经营场景」分组展示，组内按 `sort` 排序，支持拖拽调序。
- 列信息：模板名 / key / 状态徽标（草稿 · 已发布 · 已停用） / 当前发布版本 / 更新人 / 更新时间 / 近 7 天调用次数与降级率。
- 顶部操作：新建模板 / 导入 YAML / 导出 YAML。
- 行内操作：编辑 / 试跑 / 发布 / 停用 / 版本 / 复制 / 删除。
- 筛选：场景、状态、关键词。

页面二：模板编辑器 `/organize/templates/:key`

左侧分区表单（与 §5.2 的 `spec` 一一对应）：

#	分区	字段
1	基础信息	名称 / key（创建后只读） / 场景 / 图标 / 说明 / 排序
2	触发条件	入口 / 适用记忆类型 / 条数上下限 / 适用角色
3	输入装配	取数方式 / 筛选条件 / 排序 / 字数预算
4	指令	system、task 文本；变量只能从白名单下拉插入，不允许手敲 <code>{{}}</code>
5	模型策略	模型类型 / 指定模型 / temperature / max_tokens / 超时重试
6	结构要求	格式 / 产出目标 / 标题模板 / 必含章节 / 字数区间 / 禁用词 / 引用要求
7	降级策略	保底方式 / 提示文案 / 骨架文件

右侧常驻**试跑面板**（模板质量的关键保障）：

- 选 1-5 条真实记忆 → 试跑；
- 展示四段结果：① 渲染后的指令全文（可核对变量替换是否如预期）② 模型原始输出 ③ 门禁校验逐项明细 ④ 耗时与 token 消耗；
- 试跑**不落库**：不生产物、不影响线上。

顶部：草稿状态提示 / 「保存草稿」 / 「发布」 / 「版本」。

页面三：版本与回滚

- 版本列表：版本号 / 变更说明 / 发布人 / 发布时间 / 操作（查看 / diff / 回滚到此版本）。
- diff 视图：两个版本的 spec 结构化对比，指令文本按行高亮差异。
- 回滚语义：以历史版本内容新建一个发布版本，历史记录不删除（保证可审计）。

页面四：效果看板

- 按模板汇总：调用次数 / 成功率 / 降级率 / 平均耗时 / 平均 token / 采纳率（用户未编辑直接使用）。
- 按版本对比：同一模板不同版本的成功率与采纳率 —— 用来判断某次指令改动到底是不是变好了。

与前台的关系

- 后台改的是草稿，前台读的是已发布快照。后台编辑期间，前台行为完全不变。
- 停用一条模板后，前台选择器里立刻不再出现它；此前已生成的产物不受影响。

10.5 产物查看视图 ("便于自己查看"的落点)

产物不能躺在任务详情里等用户点进去翻 —— 它必须成为一个可扫、可筛、可跳的视图。以「教研提炼」为例：

我的整理	[全部场景 ▼]	🔍 搜索
筛选：	标签 [教研×] [磨课] [环创]	时间 [近 30 天 ▼]
▶ 中班"影子"磨课提炼	2026-09-18	
主题：儿童视角的意外发现		
结论 4 条 · 待办 3 项 · 来源 5 条记忆		
▶ 大班户外自主游戏观察复盘	2026-09-16	
主题：规则生成与冲突解决		
结论 3 条 · 待办 1 项 · 来源 4 条记忆		

三条设计取向

- 主字段决定"扫"的效率：列表上不铺正文，只放 summary_field + 计数型字段（结论数 / 待办数 / 来源数），让用户一眼判断"这条是不是我要找的"。
- 筛选状态可保留：整理产物的使用方式是"反复回看"，筛完标签点开一条、返回后筛选不能丢。
- 锚点即入口：正文里的 [M1] 可点，直接跳回原始记忆；记忆已删则灰显但保留编号，不报错（FR-20）。

反面案例（要避免的形态）：把整理结果做成一个"报告详情页"，进去是一篇从头读到尾的长文，没有筛选、没有字段、没有回跳。那等于把"便于自己查看"又做回了现状。

10.6 路由与深链

10.6.1 现有机制（设计必须长在上面，不能另起一套）

事实	位置
整理路由挂在 <code>/platform</code> 的 children 下，全部指向同一个组件 <code>OrganizeWorkspace.vue</code> ，靠 <code>meta.organizeTab</code> / <code>meta.memoryAsset</code> 区分页签	<code>frontend/src/router/index.ts:141-163</code>
页签清单收敛为常量数组， <code>map</code> 成路由表；route name 集中在 <code>ORGANIZE_ROUTE_NAMES</code>	<code>frontend/src/views/organize/organizeRoutes.ts:36-94</code>
<code>/platform/organize</code> 是 redirect，用 <code>?tab=</code> query 经 <code>resolveOrganizeRoutePath()</code> 落到具体页签	<code>organizeRoutes.ts:112-125</code>
独立路由只有编辑器一条： <code>organize/editor/:documentType/:id</code>	<code>router/index.ts:159</code>
服务页对照：单路由 <code>/platform/service</code> ，创建弹窗不占路由	<code>router/index.ts:165</code> 、 <code>views/service/ServiceCreateDialog.vue</code>

10.6.2 页签路由（改造 `ORGANIZE_MENU_ROUTES`）

route name	path	<code>meta.organizeTab</code>	页面
<code>organizeHub</code>	<code>/platform/organize/hub</code>	<code>hub</code>	工作台（默认入口）：上「我的整理」/ 下「整理模板」（§10.3）
<code>organizeMemory</code>	<code>/platform/organize/memory</code>	<code>memory</code>	记忆（ <code>memory/notes</code> 、 <code>memory/audio</code> 、 <code>memory/audio-cards</code> 三个资产子路由原样保留）
<code>organizeMine</code>	<code>/platform/organize/mine</code>	<code>mine</code>	我的整理：产物列表 + 字段筛选 + 排序（§10.5）
<code>organizeDiscover</code>	<code>/platform/organize/discover</code>	<code>discover</code>	发现（分类来源改为模板 scene 聚合，响应结构不变）

`router/index.ts` 的 `ORGANIZE_MENU_ROUTES.map(...)` 与 `ORGANIZE_MEMORY_ASSET_ROUTES.map(...)` 逻辑不动，只改 `organizeRoutes.ts` 的常量——这是把改动收敛在一个文件里的前提。

10.6.3 二级路由（本期仅新增两条）

route name	path	组件	说明
organizeConfigDetail	/platform/organize/configs/:configId	OrganizeConfigDetail.vue	整理详情：任务时间线（§10.3 上段的落点），头部「← 返回工作台」「设置」「+ 再次整理」
organizeOutputDetail	/platform/organize/outputs/:outputId	OrganizeOutputView.vue (详情态)	产物查看视图（字段区 + 正文 + [Mx] 锚点)

- 从时间线点「查看产出」跳 organizeOutputDetail 时带 ?from=config&configId=xxx，关闭产物后能回到正确的时间线位置，而不是退回列表页。
- 二级页与页签共用 organizeRouteMeta （requiresInit + requiresAuth），不额外引入权限分支。

10.6.4 深链约定（参数走 query，不进 path）

深链	用途
/platform/organize/hub?config=new&template={key}	从模板直接打开「新建整理」配置弹层（可分享、可从服务页/发现页引导跳入）
/platform/organize/configs/:id?job={jobId}	时间线上高亮并定位某条任务（从通知、消息卡片跳回时用）
/platform/organize/memory?select={id,id,...}&intent=organize	从外部带入预勾选记忆，落地即弹「表达要求」弹层（§10.2）
/platform/organize/mine?fields[标签]={tag}&sort={field}	产物列表的筛选/排序可分享、可刷新保持（FR-18 的 URL 化）

路由参数一律只做定位，不做状态：打开弹层是"落地动作"，关闭弹层不回写 URL，避免出现"URL 停在编辑态但界面已关闭"的刷新歧义。

10.6.5 redirect 落点（需拍板，见 Q11）

`resolveOrganizeRoutePath()` 的兜底从 `memory` 改为 `hub` —— 工作台是新信息架构的入口，`/platform/organize` 和 `?tab=` 的旧链接应落到工作台。

风险：老用户书签 `/platform/organize` 的落点会从「记忆」变成「工作台」。保守替代方案是保持 `memory` 不动，把工作台仅作为页签之一（首屏便利性下降）。这是本设计里唯一影响存量用户习惯的点。

10.6.6 刻意不给路由的部分

不路由	理由
整理配置弹层（新建 / 编辑）	对齐服务页 <code>ServiceCreateDialog</code> 的既有做法：弹窗是瞬态，路由化会产生"关闭后 URL 仍在编辑态"的刷新歧义；分享需求由 §10.6.4 的 <code>?config=new</code> 覆盖
「表达要求」弹层（§10.2）	它依赖"已选记忆"这一瞬时上下文，URL 化会暴露一串裸 memory id，可读性与安全性都差
进度弹层（§10.3 执行中）	进度是任务态而非页面态；刷新后应回到时间线看任务状态（ <code>?job=</code> 高亮），而不是恢复一个弹窗

10.6.7 后端与后台路由

- API：沿用 PRD §9 的 `/organize/...` 前缀 + 角色守卫，**不新开 `/admin` 前缀**（与全仓惯例一致，仅 `internal/router/router.go:1293` 一处例外）；配置实例一组接口见 §9.2.2。
- 后台页面：在 `admin/src/router` 注册 `organize-templates`（列表）、`organize-templates/:key`（编辑器）、`organize-templates/:key/versions`（版本），并加入 `admin/src/config/navigation.ts` 的 `ADMIN_NAV_GROUPS` 「资产治理」分组（`requiresSystemAdmin: true`）。组件经 `@` → `../frontend/src` 别名复用前台 API 层，不复制一份。

对应验收：**AC-21**（深链可用）、**AC-22**（弹窗关闭后 URL 与界面状态一致）。

11. 兼容与迁移

11.1 迁移阶段

阶段	动作	校验
S1	建表 000101 + 模板引擎与加载器 + 把 4 条存量提示词整理成 seed YAML，不接线上链路	单测：4 条 seed 模板从库中读出并渲染后，与现有硬编码 prompt 逐字节一致
S2	4 条存量 AI 链路切到模板（读已发布快照）， <code>organize.template_engine.enabled</code> 开关控制	同输入下产物差异 = 0（固定记忆集回归）
S3	后台模板管理上线，平台开始维护；开启批量整理与场景模板（新功能，不影响存量）	后台改草稿 → 发布 → 前台生效；停用 → 前台选择器消失
S4	关闭旧代码路径，删除硬编码提示词	全量对比

冷启动说明：升级完成后库里没有任何模板，前台会短暂"无模板可用"。所以 S1 必须把建表与 seed 放在同一次发布里 —— 迁移完成 → 服务启动 → seed 写入 4 条模板（`status: enabled`、`is_seed: true`）→ 前台立即可用。

11.2 回退方案

- 开关回退：`config.yaml` 中 `organize.template_engine.enabled: false` 即整体回到旧路径。
- 模板级回退：单条模板 `status: disabled` 后，该 key 的请求自动落到内置基线或旧路径。
- 数据回退：新增字段均有默认值，Drop 新表不影响存量数据读取。

11.3 迁移风险点

风险	缓解
模板渲染与硬编码 prompt 不一致（多空行、 <code>\n</code> 差异）	S1 阶段做逐字节 diff 断言，作为合并门槛
存量产物 <code>template_key</code> 为空导致前端 NPE	前端一律按"历史内容"降级展示，接口返回空串而非 null
存量产物没有 <code>fields</code> ，与新产品混排时筛选器出现"空桶"	查看视图对 <code>fields = {}</code> 的产物走"历史内容"分支：不参与字段筛选，但可按时间排序、可全文搜索
升级后库里没有模板，前台无模板可选	建表与 seed 同一次发布；seed 失败不阻塞启动，但打 ERROR 告警并触发监控
种子把运维改过的模板打回	种子只插入缺失 key，从不 UPDATE； <code>is_seed</code> 仅作来源标记
后台误改、误删导致线上异常	草稿与发布分离 + 发布前必试跑 + 变更说明必填 + 版本可回滚 + 可一键停用
GIN 索引在 MySQL 方言不可用	MySQL 侧退化为前缀索引 + 应用层过滤，接口语义不变

12. 分期实施

期	内容	交付物	依赖
P0	建表 000101 + 「整理要求」抽象与落定层 + 模板引擎 + Schema 校验 + 初始化种子 + 4 条存量链路等价迁移（开关灰度）	行为完全不变，能力就位	—
P1	后台模板管理：列表 / 编辑器 / 试跑 / 发布 / 停用 / 版本回滚 + 导航项	平台可自助维护模板	P0
P2	批量整理 + 8 条场景模板（后台创建）+ 模板选择接口 + 前台选择器	用户可用新整理能力	P1
P3	整理工作台 + 进度 SSE + 产物查看视图（可筛可跳）+ facets 接口 + 效果看板 + YAML 导入导出	完整体验闭环	P2
P4	自动整理规则（转写后自动跑、按 tag 自动归档）	自动化沉淀	P3

分期节奏的关键变化（v1.1）：原计划把后台管理放 P2、先做用户侧能力。既然模板真源就在后台，"平台能建模板"是用户侧能力的前置条件——否则用户功能上线了，模板还得靠研发写 SQL 往里塞。所以后台管理提到 P1，用户侧能力顺延到 P2。

建议 P0 单独立项：P0 是纯内部重构，对用户零感知，可独立合并、独立回归、独立回退（一个开关）。P0 落地是 P1 的开工前提。

v1.2 补充：「整理要求」抽象必须在 P0 就位，不能等到做前台时再补。原因是它是引擎的入参类型——若 P0 直接从 `template_key` 起步，P2 做用户侧时必然要把入参从 `template_key` 改成 `Requirement`，等于把 P0 的引擎与 P1 的后台一起返工。先定入参类型，代价只是多一层薄封装。

若 Q8 定为"要做自定义要求"：新增 P3.5 一期，产出"自由描述 → 要求解析 → 歧义追问 → 落定"的解析器与对话式确认界面，依赖 P3（因为需要效果数据来判断哪些自由描述高频出现、值得固化成新模板）。

v1.3 补充（"便于自己查看"落到哪一期）：

能力	期	理由
<code>fields</code> / <code>citations</code> 两列 + 字段派生（V1）+ 锚点解析（V3）	P0	它们动的是同一张产物表，必须在同一次迁移里落地；但 P0 只做 <code>derived</code> 字段与锚点解析，不改存量提示词，所以 4 条链路行为等价（D14）依旧成立
新场景模板必须声明 <code>fields</code> ，V1 成为发布门槛	P2	与新模板一起推，靠 Schema 校验拦住（FR-22）
产物查看视图 + facets 接口	P3	到这一期"便于自己查看"才对用户可感——前两期是能力就位

13. 风险与验收标准

13.1 风险

编号	风险	等级	缓解
R1	模板过多导致选择困难	中	场景分组 + tag 命中推荐 + <code>sort</code> 权重
R2	自由指令带来的输出不稳定	高	变量白名单 + 输出契约 + 门禁自修复
R3	批量输入导致 token 成本上升	中	<code>max_total_runes</code> 预算 + token 用量落 job
R4	平台误改生产模板	高	草稿态 + 试跑 + 发布二次确认 + 版本回滚 + 变更留痕
R5	与 Agent 模块的整理能力职责重叠	中	明确：整理模块面向"记忆 → 结构化成果"的确定性场景；需要多步推理 / 工具调用走 Agent 模块
R6	平台是唯一维护方，无二审，存在单点风险	中	本期靠"草稿/发布分离 + 版本回滚 + 变更说明必填 + 启用开关"兜底；模板数量上量后再考虑双人复核（见 Q6）
R7	模板全部平台统一，照顾不到单个园所的差异	中	本期接受：先验证哪些场景真的需要差异化； <code>visibility</code> 字段已预留 <code>tenant</code> 层，将来不必改表
R8	为满足"可筛字段"而逼模型编造字段值	高	优先用 <code>derived</code> 字段兜住 V1； <code>source: model</code> 的字段在门禁里校验"抽不到值即违规"，且模板约束中显式禁止编造缺失信息（信息不足填"待补充"）
R9	锚点在记忆被删除后失效，跳转报错打断阅读	中	锚点解析接口对已删记忆返回 <code>missing: true</code> ，前端灰显并保留编号，不报错（FR-20 / AC-19）
R10	字段一旦发布即成为下游依赖，后期改字段语义会破坏筛选与看板	中	字段 <code>key</code> 发布后不可改（同模板 <code>key</code> ）；只允许新增字段或改 <code>label</code> ，语义变更须新建字段

13.2 验收标准

编号	标准	验证方式
AC-1	AI 整理相关提示词在 Go 代码中 0 处字面量	<code>grep</code> 断言 + Code Review
AC-2	4 条存量链路在固定记忆集下产物与旧版一致	回归测试逐字节对比
AC-3	后台新增一条场景模板，无需发版即可被选择器命中	手工验收
AC-4	模板指令改动可试跑、可发布、可回滚到任意历史版本	手工验收
AC-5	不合规输出 100% 被门禁拦截并被修复或降级	注入违规模板做故障演练
AC-6	每个产物可查到 <code>template_key + version + prompt_hash + model_id + memory_ids</code>	数据库抽查
AC-7	整理任务状态机覆盖全部 7 态，SSE 进度可视	前端联调
AC-8	前端不再硬编码 8 个场景分类	Code Review
AC-9	开关关闭后系统行为与重构前完全一致	灰度回退演练
AC-10	平台管理员在后台新建 / 修改 / 停用模板，全程无需发版、无需重启服务	手工验收
AC-11	后台改草稿期间，前台行为与改前完全一致（发布才生效）	灰度演练
AC-12	模板可回滚到任意历史版本，回滚后运行期行为回到该版本	手工验收
AC-13	库中存在非法模板时服务仍能正常启动，非法模板被自动停用并告警	故障注入
AC-14	后台「整理模板」入口仅平台系统管理员可见，普通租户管理员看不到	权限用例
AC-15	每个 job 都存有落定后的要求快照；修改模板后重跑历史 job，其行为与该 job 创建时一致（不随模板漂移）	数据库抽查 + 回归用例
AC-16	前台允许「模板」一词（对齐服务页"从模板创建"语汇），但不得出现"提示词""指令""参数"等实现词；卡片文案只描述产出	文案走查
AC-17	每个新产物的列表项在 不打开正文 的情况下即可判断主题与规模（主字段 + 计数型字段齐全）	界面走查
AC-18	产物可按字段筛选与排序，且从产物详情返回后筛选状态保留	前端联调
AC-19	产物中的 <code>[Mx]</code> 锚点可跳回原始记忆；对应记忆被删除后灰显保留编号、不报错	用例
AC-20	未声明 <code>fields</code> 的模板无法发布；已发布模板必定带 ≥ 2 个可筛选字段	后台用例 + 数据库抽查

编号	标准	验证方式
AC-21	4 条页签路由与 2 条二级路由可直达（刷新/粘贴链接即落在正确页面）； <code>config=new&template=</code> 能直接唤起配置弹层	前端用例
AC-22	关闭弹层后 URL 与界面状态一致（不出现 URL 停在编辑态而界面已关闭）；产物详情返回后回到来源页（时间线或列表）	前端用例

14. 待决策问题

以下 9 项需在评审时拍板，会直接影响实现路径：

14.1 已结题（v1.1）

编号	问题	结论	连带影响
Q1	模板由谁维护？	由平台在 admin 后台维护 和 创建 （平台系统管理员，入口挂「资产治理」）	真源从 YAML 改为 DB；解析优先级简化为单层；后台管理从 P2 提前到 P1
Q2	普通用户 / 租户能否自建模板？	不能。本期 visibility 固定 platform ， tenant / personal 仅作字段预留	FR-15 移出本期；§8.4 改写为"平台模板治理"

14.2 待决策

编号	问题	建议方案	影响面
Q3	模板粒度按场景还是按产物形态？	按场景 （招生 / 家长服务 / 教研...），产物形态作为模板属性	决定后台列表与前台选择器的信息架构
Q4	是否需要"自动整理"（转写后自动发芽）？	放 P4 ，先观察手动整理的模板命中分布	决定 <code>trigger.auto_run</code> 是否本期启用
Q5	模板效果看板的指标口径？	首期仅： 成功率 / 降级率 / 平均耗时 / 用户采纳率 （未编辑直接使用）	决定埋点范围
Q6	平台单人维护是否需要双人复核？	本期不做 ；等模板数量上量（>20 条）或出现一次误操作事故后再引入	决定后台是否要加审批状态机
Q7	8 条场景模板的指令由谁起草？	平台运营起草 + 教研审内容 ，研发只负责把字段做齐	决定 P2 的内容准备工作由谁承担
Q8	"用户的要求"是否包含 自由描述 （自定义要求）？	本期不做 ，只做预置要求的表达（选模板 + 选范围）；引擎的 <code>requirement.provider</code> 预留 <code>custom</code> 插槽，等有真实需求数据再开	这是 v1.2 一句话定义带出的新问题。若开，需额外设计"自由文字 → 结构化要求"的落定与歧义消解（追问对话），工作量约等于再加一个模块；若不开，定义里的"用户的要求"由"选模板 + 定范围"承载， 同样成立
Q9	「成果」页签是否也严格限定"给自己看"？	先按 self 处理 ：本期所有产物受众固定 <code>self</code> ，需要对外展示的内容由用户自己另存/导出	定义说"便于 自己 查看"，但园所语境里「成果」常被理解成可对外展示的成果集（给家长看、给评估看）。若「成果」实际要承担对外展示，那它需要独立的 <code>audience: others</code> 模板与措辞规范，本期的"不做对外材料"边界就要改
Q10	是否允许配置声明「默认整理」，使某类记忆 零选择直通 ？	默认关闭 ，先观察误触率再放开	见《整理助理交互调研》§4：Tana / Mem 的静默全自动路由在园所场景对"口径正确"有风险；关闭时按"唯一命中直通 + 多命中轻菜单 + 零命中引导"三路处理（排期 P2）
Q11	<code>/platform/organize</code> 的 redirect 兜底落点改为「工作台」还是保持「记忆」？	改为工作台 （新信息架构的入口）	唯一影响存量用户习惯的点：老书签落点会从「记忆」变成「工作台」。若评审认为不可接受，则保持 <code>memory</code> ，工作台只作为页签之一（§10.6.5）

附录 A：与现有代码的对应关系速查

现状位置	重构后去向
<code>organize_sprout.go:170-179</code> (system prompt)	<code>seed/sprout_review.yaml</code> → <code>instructions.system</code>
<code>organize_sprout.go:193-226</code> (<code>buildSproutAIPrompt</code>)	<code>seed/sprout_review.yaml</code> → <code>instructions.task</code>
<code>organize_sprout.go:228-281</code> (fallback 骨架)	<code>fallbacks/sprout_review.md</code> + <code>fallback.hint</code>
<code>organize_sprout.go:382-409</code> (角色标签 / 配置)	模板变量 <code>tenant.role_label</code> / <code>tenant.role_config</code>
<code>organize_upload.go:227-242</code> (导入元数据 prompt)	<code>seed/note_import_meta.yaml</code>
<code>organize_upload.go:292-310</code> (录音笔记 prompt)	<code>seed/note_audio_transcribe.yaml</code>
<code>organize_upload.go:363-378</code> (成果卡片 prompt)	<code>seed/output_card_meta.yaml</code>
<code>organize_upload.go:436-471</code> (JSON 解析)	提升为通用 <code>renderer</code> + <code>json_schema</code> 校验
<code>organize_upload.go:413-434</code> (<code>resolveOrganizeModelID</code>)	保留，作为模板 <code>model.model_id</code> 为空时的兜底
<code>organize_sprout.go:16</code> (<code>organizeSproutPromptRuneBudget</code>)	<code>input.budget.max_total_runes</code> (默认 7000)
<code>types/organize.go:217-239</code> (8 分类常量)	模板 <code>scene</code> 字段 + <code>/organize/templates/scenes</code>
<code>frontend/.../discoverCategories.ts</code>	改为接口拉取，保留兜底常量
<code>frontend/src/views/organize/organizeRoutes.ts:36-94</code> (3 个页签)	扩为 4 个页签 (新增 <code>hub</code> / 重命名 <code>mine</code>)，并新增 <code>configDetail</code> / <code>outputDetail</code> 两个 route name
<code>router/index.ts:141-163</code> (整理路由注册)	逻辑不变，仅随常量新增两条二级路由 (见 §10.6.3)
<code>migrations/versioned/000072</code>	新增 <code>000101_organize_templates</code>
<code>admin/src/config/navigation.ts</code> (「资产治理」分组)	新增「整理模板」导航项 (<code>requiresSystemAdmin</code>)

附录 B：模板编写规范（给平台运营的一页纸）

- 一条模板解决一个场景。不要把"招生线索跟进"和"渠道效果分析"塞进一条模板。
- 指令里只写"要什么"，不写"怎么取数"。取数由「输入装配」配置决定。
- 必须声明输出结构。至少给出「必含章节」，否则模型会自由发挥。

4. **必须声明「可筛字段」**。哪怕模型产不出，也要用 `derived` 兜住 —— 产物要"便于自己查看"，字段是承重墙，没有字段的模板不允许发布。
5. **尽量要求引用锚点**（ `citation: required` ）。每条结论带 `[Mx]`，用户才能从整理结果一跳回到原始记录。
6. **必须写明不许编造**。显式声明"不得编造记忆中没有的数字、人名、园所名"。
7. **变量从下拉里插，不要手敲**。手敲的未声明变量会让整理任务直接失败。
8. **发布前必须试跑**，且至少用 3 条不同类型的记忆试跑，确认渲染后的指令读起来是对的。
9. **改指令必须写变更说明**。版本记录会留痕，出问题时才能快速定位和回滚。
10. **不用了先停用，别急着删**。停用立即对用户生效，删掉就再也对不上了。